SmartCity360

Smart City Management Platform – OODP II Project Documentation

Java 17 · Jakarta EE 10 · JSF · EJB · JPA · MySQL · Payara 6

Assessment alignment

Part	Required scenario	Pattern	Implementation
A	Dynamic security wrappers	Decorator	DataStream + authentication, encryption and compression decorators
B	Hierarchical smart zones	Composite	CityComponent, ZoneGroup and Building
C	Escalation of complaints	Chain of Responsibility	Local staff → supervisor → city manager
D	Memory-efficient streetlights	Flyweight	Shared LightType factory + per-location context
E	City policy mini-language	Interpreter	Comparison and AND expressions with PolicyParser

1. Introduction and objectives

SmartCity360 is a web-based city management prototype for organizing city zones, energy readings, citizen complaints, streetlight configurations and operational policies. The project applies the five patterns specified by the Java Institute OODP II assessment. Its objectives are to demonstrate maintainable object-oriented design, separate presentation/business/persistence responsibilities, and provide a working basis for a VIVA demonstration.

2. Architecture and technology

The application is a Maven multi-module EAR. The JSF web module contains XHTML pages and CDI backing beans. The EJB module contains JPA entities, stateless services and design-pattern packages. The EAR packages the WAR and EJB module for Payara Server. MySQL stores application records through a JTA datasource named jdbc/SmartCityDS.

3. Part A – Decorator

DataStream defines a common process(String) contract. BasicEnergyStream is the concrete component. StreamDecorator wraps another DataStream. AuthenticationDecorator, EncryptionDecorator and CompressionDecorator add independent processing layers. EnergyService composes the wrappers without changing the basic stream. Benefits include reusable layers and flexible combinations. The order of wrappers matters and excessive wrapping can make debugging harder. Important: Base64 is encoding, not real encryption; the authentication wrapper is only a demonstration. Production use requires real authentication, authenticated cryptography and TLS.

4. Part B – Composite

CityComponent defines getName and execute. Building is a leaf; ZoneGroup is a composite containing child CityComponents. A command can be sent to a group and propagated recursively to its descendants. This supports city → district → street → building structures and uniform operations. Validation is needed if certain node types must only contain specific child types. Zone persistence uses parentId, while the Composite package demonstrates the in-memory hierarchy.

5. Part C – Chain of Responsibility

ComplaintHandler defines the handler link and handle operation. LocalStaffHandler handles severity 1–2; SupervisorHandler handles 3–4; CityManagerHandler handles 5 and above. Each handler either resolves the assignment or forwards the complaint. This decouples submission from the final handler and allows the chain to be reordered. Requests can be difficult to trace without logging, and an incomplete chain needs a fallback.

6. Part D – Flyweight

LightType stores shared intrinsic state: model, wattage and colour. LightTypeFactory caches and reuses LightType objects. StreetLightContext stores extrinsic state such as location and on/off state. This avoids duplicating identical configuration objects across thousands of streetlights. The shared flyweight should remain immutable; the design introduces a distinction between shared and per-light state.

7. Part E – Interpreter

Expression defines interpret(PolicyContext). ComparisonExpression evaluates one variable/operator/number expression. AndExpression combines two expressions. PolicyParser parses examples such as noiseDb > 60 AND hour >= 22. The sample context noiseDb=65 and hour=22 evaluates to true. The supported grammar is intentionally small: one comparison or two comparisons joined by AND. A larger DSL would require a more capable parser and grammar design.

8. Data model

Zone stores a name, type and parentId. Complaint stores resident, description, severity, assigned level, status and creation time. Streetlight stores code, location, model and active state. EnergyReading stores meter code, kWh and timestamp. CityPolicy stores a policy name and expression.

9. Build, database and deployment

Run `mvn clean package` from the root folder. Execute database/smartcity360.sql in MySQL. Configure a Payara JDBC connection pool using com.mysql.cj.jdbc.MysqlDataSource and create the JNDI resource jdbc/SmartCityDS. Install MySQL Connector/J in the Payara domain library and restart the server. Deploy smartcity-ear/target/SmartCity360.ear. The default page is /smartcity-web/pages/dashboard.xhtml. Use the actual context root displayed by Payara if it differs.

10. Test plan

Create a city hierarchy and verify parent IDs. Submit complaint severities 1, 3 and 5 and verify their assigned levels. Register streetlights with a shared model and run the Flyweight demo. Process an energy sample and inspect the decorated output. Evaluate noiseDb > 60 AND hour >= 22 with noiseDb=65 and hour=22; expected result is true. Also test invalid expressions and required fields.

11. VIVA demonstration (10–15 minutes)

0–1 min: problem and architecture. 1–3 min: dashboard and zones/Composite. 3–5 min: complaint escalation/Chain. 5–7 min: streetlights/Flyweight. 7–9 min: policy evaluation/Interpreter. 9–11 min: energy stream/Decorator. 11–13 min: UML, database and source packages. 13–15 min: benefits, limitations and future improvements.

12. Limitations and future work

This is an academic starter implementation. Add authentication and role-based authorization, audit logging, stronger validation, real IoT ingestion, real cryptography, a richer policy grammar, recursive loading of persisted zones, pagination, automated tests and secure externalized database credentials before production use.

13. Conclusion

SmartCity360 brings the five required design patterns together in a coherent city-management scenario. The multi-tier architecture separates the web interface, business services and persistence, while the pattern packages demonstrate extensibility, decoupling, hierarchical composition, shared memory and rule interpretation.

Appendix A – UML class diagram source

PlantUML source files are included in documentation/uml. Open each .puml file with a PlantUML renderer to generate the corresponding class diagram.

PartA-Decorator.puml

```
@startuml
interface DataStream { +process(data:String):String }
class BasicEnergyStream
abstract class StreamDecorator { -wrapped:DataStream }
class AuthenticationDecorator
class EncryptionDecorator
class CompressionDecorator
DataStream <|.. BasicEnergyStream
DataStream <|.. StreamDecorator
StreamDecorator <|-- AuthenticationDecorator
StreamDecorator <|-- EncryptionDecorator
StreamDecorator <|-- CompressionDecorator
StreamDecorator o--> DataStream
@enduml
```

PartB-Composite.puml

```
@startuml
interface CityComponent { +getName():String +execute(command:String):void }
class ZoneGroup
class Building
CityComponent <|.. ZoneGroup
CityComponent <|.. Building
ZoneGroup o--> "0..*" CityComponent : children
@enduml
```

PartC-Chain.puml

```
@startuml
abstract class ComplaintHandler { -next:ComplaintHandler +setNext(next):ComplaintHandler +handle(c):void }
class LocalStaffHandler
class SupervisorHandler
class CityManagerHandler
ComplaintHandler <|-- LocalStaffHandler
ComplaintHandler <|-- SupervisorHandler
ComplaintHandler <|-- CityManagerHandler
ComplaintHandler --> ComplaintHandler : next
@enduml
```

PartD-Flyweight.puml

```
@startuml
class LightType { -model:String -wattage:int -colour:String }
class LightTypeFactory { +get(model,wattage,colour):LightType }
class StreetLightContext { -location:String -on:boolean -sharedType:LightType }
LightTypeFactory ..> LightType : creates/caches
StreetLightContext --> LightType : shared intrinsic state
@enduml
```

PartE-Interpreter.puml

```
@startuml
interface Expression { +interpret(context):boolean }
class ComparisonExpression
class AndExpression
class PolicyContext
Expression <|.. ComparisonExpression
Expression <|.. AndExpression
AndExpression o--> Expression : left/right
Expression ..> PolicyContext
@enduml
```